

Operating System

Name: Nget Darapich

Id: e20221646

I4-Group-B

Activity: Synchronization II Semaphore

Problem1

```

1 import threading
2 import time
3 import random
4
5 # --- Shared Memory and Parameters ---
6 BUFFER_SIZE = 100
7 buffer = [None] * BUFFER_SIZE
8 current_write_pos = 0
9 current_read_pos = 0
10
11 # --- Semaphores and Initial Values ---
12 S = threading.Semaphore(0) # full_pairs: starts at 0
13 space = threading.Semaphore(50) # empty_spaces: room for 50 pairs
14 lock = threading.Semaphore(1) # mutex: ensure consecutive storage
15
16 def producer(machine_id):
17     global current_write_pos
18     while True:
19         try:
20             # Produce pair P1, P2
21             p1 = f"M{machine_id}-P1"
22             p2 = f"M{machine_id}-P2"
23
24             # Rule 1 & 3: Check if 2 spaces are available
25             if not space.acquire(timeout=2):
26                 print(f"ERROR: Machine {machine_id} timed out waiting for buffer space. (Buffer Full)")
27                 continue
28
29             # Rule 2: Enter critical section for consecutive placement
30             lock.acquire()
31
32             # Double-check buffer bounds before writing (Error Prevention)
33             if buffer[current_write_pos] is not None:
34                 print(f"CRITICAL ERROR: Overwriting data at index {current_write_pos}")
35
36             print(f"Machine {machine_id} placing pair at [{current_write_pos}, {current_write_pos+1}]")
37             buffer[current_write_pos] = p1
38             buffer[current_write_pos + 1] = p2
39             current_write_pos = (current_write_pos + 2) % BUFFER_SIZE
40
41             lock.release()
42             S.release() # Signal consumer that pair is ready
43
44             time.sleep(random.uniform(0.1, 0.5))
45
46         except Exception as e:
47             print(f"SYSTEM ERROR in Producer {machine_id}: {e}")
48             if lock.locked(): lock.release() # Prevent deadlock on failure
49
50 def consumer():
51     global current_read_pos
52     while True:
53         try:
54             # Rule 4: Packaging machine breaks if it finds buffer empty
55             # wait(5) ensures we only proceed when at least 2 particles exist
56             if not S.acquire(timeout=5):
57                 print("ERROR: Consumer timed out. (Buffer Empty/System Idle)")
58                 continue
59
60             # Error checking for valid pair data
61             p1 = buffer[current_read_pos]
62             p2 = buffer[current_read_pos + 1]
63
64             if p1 is None or p2 is None:
65                 raise ValueError(f"Consumer found corrupted data at index {current_read_pos}")
66
67             print(f"Consumer fetched: ({p1}, {p2})")
68
69             # Clean buffer after fetching (Simulation of removal)
70             buffer[current_read_pos] = None
71             buffer[current_read_pos + 1] = None
72
73             current_read_pos = (current_read_pos + 2) % BUFFER_SIZE
74
75             space.release() # Signal that 1 pair space is now free
76
77             # Package and ship
78             time.sleep(0.4)
79
80         except ValueError as ve:
81             print(f"DATA ERROR: {ve}")
82         except Exception as e:
83             print(f"SYSTEM ERROR in Consumer: {e}")
84
85 # Start Simulation
86 for i in range(1, 4):
87     threading.Thread(target=producer, args=(i,)).start()
88     threading.Thread(target=consumer, daemon=True).start()
89
90 time.sleep(5)
91 print("Simulation monitor finished.")

```

result in terminal powershell

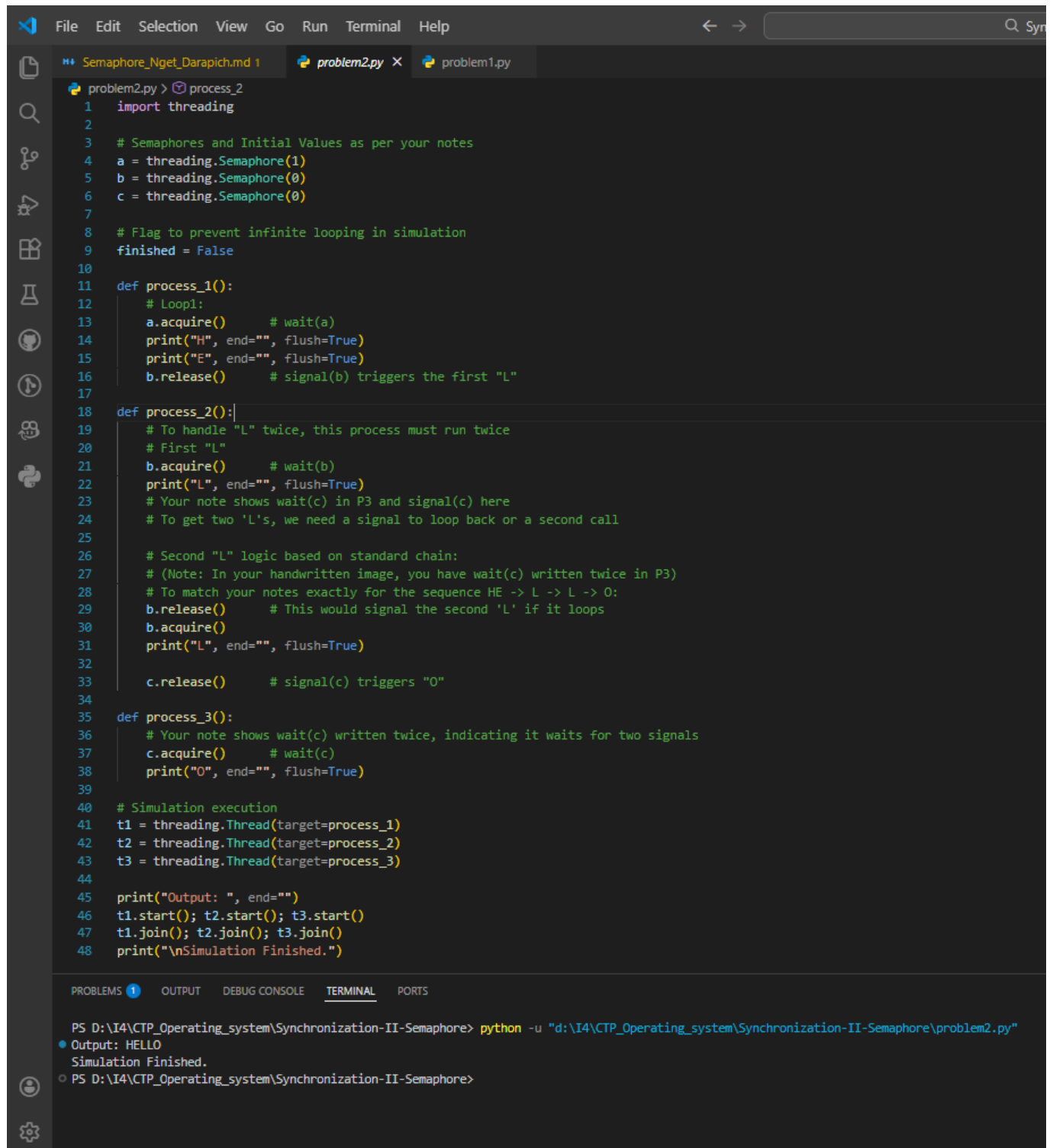
```

PS D:\I4\CTP_Operating_system\Synchronization-II-Semaphore> & "C:\Users\MSI
15\AppData\Local\Programs\Python\Python312\python.exe"
d:/I4/CTP_Operating_system/Synchronization-II-Semaphore/problem1.py
Machine 1 placing pair at [0, 1]
Machine 2 placing pair at [2, 3]
Machine 3 placing pair at [4, 5]
Consumer fetched: (M1-P1, M1-P2)
Machine 1 placing pair at [6, 7]
Machine 2 placing pair at [8, 9]
Machine 1 placing pair at [10, 11]
Machine 3 placing pair at [12, 13]
Consumer fetched: (M2-P1, M2-P2)
Machine 2 placing pair at [14, 15]
Machine 1 placing pair at [16, 17]
Machine 3 placing pair at [18, 19]

```

```
Consumer fetched: (M3-P1, M3-P2)
Machine 1 placing pair at [20, 21]
Machine 2 placing pair at [22, 23]
Machine 3 placing pair at [24, 25]
Machine 1 placing pair at [26, 27]
Machine 3 placing pair at [28, 29]
Consumer fetched: (M1-P1, M1-P2)
Machine 2 placing pair at [30, 31]
Machine 3 placing pair at [32, 33]
Machine 1 placing pair at [34, 35]
Machine 1 placing pair at [36, 37]
Machine 2 placing pair at [38, 39]
Consumer fetched: (M2-P1, M2-P2)
Machine 3 placing pair at [40, 41]
Machine 1 placing pair at [42, 43]
Consumer fetched: (M1-P1, M1-P2)
Machine 2 placing pair at [44, 45]
Machine 3 placing pair at [46, 47]
Machine 2 placing pair at [48, 49]
Machine 1 placing pair at [50, 51]
Machine 3 placing pair at [52, 53]
Consumer fetched: (M3-P1, M3-P2)
Machine 2 placing pair at [54, 55]
Machine 1 placing pair at [56, 57]
Consumer fetched: (M2-P1, M2-P2)
Machine 3 placing pair at [58, 59]
Machine 2 placing pair at [60, 61]
Machine 1 placing pair at [62, 63]
Machine 2 placing pair at [64, 65]
Consumer fetched: (M1-P1, M1-P2)
Machine 3 placing pair at [66, 67]
Machine 2 placing pair at [68, 69]
Machine 1 placing pair at [70, 71]
Machine 3 placing pair at [72, 73]
Consumer fetched: (M3-P1, M3-P2)
Machine 2 placing pair at [74, 75]
Machine 2 placing pair at [76, 77]
Machine 1 placing pair at [78, 79]
Consumer fetched: (M1-P1, M1-P2)
Machine 3 placing pair at [80, 81]
Machine 3 placing pair at [82, 83]
Machine 2 placing pair at [84, 85]
Machine 1 placing pair at [86, 87]
Machine 2 placing pair at [88, 89]
Consumer fetched: (M2-P1, M2-P2)
Machine 3 placing pair at [90, 91]
Machine 1 placing pair at [92, 93]
Machine 2 placing pair at [94, 95]
Consumer fetched: (M3-P1, M3-P2)
Machine 3 placing pair at [96, 97]
Simulation monitor finished.
PS D:\I4\CTP_Operating_system\Synchronization-II-Semaphore>
```

Problem2



```
File Edit Selection View Go Run Terminal Help
Semaphore_Nget_Darapich.md 1 problem2.py x problem1.py
problem2.py > process_2
1 import threading
2
3 # Semaphores and Initial Values as per your notes
4 a = threading.Semaphore(1)
5 b = threading.Semaphore(0)
6 c = threading.Semaphore(0)
7
8 # Flag to prevent infinite looping in simulation
9 finished = False
10
11 def process_1():
12     # Loop1:
13     a.acquire()      # wait(a)
14     print("H", end="", flush=True)
15     print("E", end="", flush=True)
16     b.release()      # signal(b) triggers the first "L"
17
18 def process_2():
19     # To handle "L" twice, this process must run twice
20     # First "L"
21     b.acquire()      # wait(b)
22     print("L", end="", flush=True)
23     # Your note shows wait(c) in P3 and signal(c) here
24     # To get two 'L's, we need a signal to loop back or a second call
25
26     # Second "L" logic based on standard chain:
27     # (Note: In your handwritten image, you have wait(c) written twice in P3)
28     # To match your notes exactly for the sequence HE -> L -> L -> O:
29     b.release()      # This would signal the second 'L' if it loops
30     b.acquire()
31     print("L", end="", flush=True)
32
33     c.release()      # signal(c) triggers "O"
34
35 def process_3():
36     # Your note shows wait(c) written twice, indicating it waits for two signals
37     c.acquire()      # wait(c)
38     print("O", end="", flush=True)
39
40 # Simulation execution
41 t1 = threading.Thread(target=process_1)
42 t2 = threading.Thread(target=process_2)
43 t3 = threading.Thread(target=process_3)
44
45 print("Output: ", end="")
46 t1.start(); t2.start(); t3.start()
47 t1.join(); t2.join(); t3.join()
48 print("\nSimulation Finished.")

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\I4\CTP_Operating_system\Synchronization-II-Semaphore> python -u "d:\I4\CTP_Operating_system\Synchronization-II-Semaphore\problem2.py"
● Output: HELLO
Simulation Finished.
○ PS D:\I4\CTP_Operating_system\Synchronization-II-Semaphore>
```